

Architecture Multi-tenant SaaS

Concevoir un logiciel qui sert plusieurs clients en isolation

Multi-tenant	SaaS	RLS	Isolation
--------------	------	-----	-----------

Scalabilité	Facturation
-------------	-------------

Parcours de formation — Développement d'Applications Mobiles

Djiguitech · Social Seller · Tous niveaux

■ Compétences visées

- ✓ Définir le concept de multi-tenancy et ses différentes approches architecturales
- ✓ Choisir et implémenter la stratégie d'isolation de données adaptée
- ✓ Appliquer Row Level Security pour garantir l'isolation au niveau de la base de données
- ✓ Concevoir un système de permissions et de rôles par tenant
- ✓ Comprendre les enjeux de scalabilité spécifiques au SaaS
- ✓ Planifier la facturation et la gestion des quotas par tenant

Table des matières

Chapitre 1 — Qu'est-ce qu'un SaaS multi-tenant ?

- 1.1 Définition et modèles économiques
- 1.2 Single-tenant vs multi-tenant
- 1.3 Les trois approches d'isolation
- 1.4 Choisir la bonne approche

Chapitre 2 — Isolation des données avec RLS

- 2.1 La colonne tenant_id
- 2.2 Politiques RLS multi-tenant
- 2.3 Fonctions helper pour le contexte
- 2.4 Tester l'isolation

Chapitre 3 — Gestion des identités et permissions

- 3.1 Modèle RBAC
- 3.2 Rôles par tenant
- 3.3 Permissions granulaires
- 3.4 Audit trail

Chapitre 4 — Scalabilité et performance

- 4.1 Index tenant-aware
- 4.2 Connection pooling
- 4.3 Caching par tenant
- 4.4 Monitoring multi-tenant

Chapitre 5 — Facturation et limites

- 5.1 Plans tarifaires
- 5.2 Comptage de l'usage
- 5.3 Limites et quotas

5.4 Stripe et abonnements

Qu'est-ce qu'un SaaS multi-tenant ?

1.1 Définition et modèles économiques

Un SaaS (Software as a Service) multi-tenant est un logiciel hébergé dans le cloud qui sert plusieurs clients (tenants) depuis une infrastructure partagée. Chaque tenant est une organisation distincte — dans Social Seller, chaque boutique est un tenant. Les tenants partagent le code de l'application et l'infrastructure (serveurs, base de données, files de messages) mais leurs données sont strictement isolées. Ils ne se voient jamais les uns les autres.

Le modèle économique SaaS repose sur l'abonnement récurrent et les économies d'échelle. Maintenir une seule infrastructure pour 1000 clients coûte beaucoup moins cher que 1000 déploiements séparés. Chaque nouvelle fonctionnalité est disponible immédiatement pour tous les clients. Les mises à jour sont déployées une seule fois. C'est cette efficacité opérationnelle qui rend le SaaS rentable à grande échelle.

1.2 Single-tenant vs multi-tenant

Single-tenant vs multi-tenant

Critère	Single-tenant	Multi-tenant
Infrastructure	Une instance par client	Instance partagée
Isolation	Totale (VMs séparées)	Logicielle (RLS, tenant_id)
Coût opérationnel	Élevé (N instances)	Faible (1 instance)
Personnalisation	Maximale	Limitée aux options prévues
Déploiement	Client par client	Global pour tous
Scalabilité	Difficile à coordonner	Centralisée et homogène
Marché cible	Entreprises (compliance)	PME, startups

1.3 Les trois approches d'isolation

Il existe trois façons d'isoler les données dans un SaaS multi-tenant, avec des compromis différents entre isolation, coût et complexité.

Approche 1 — Base de données séparée par tenant : isolation maximale, migration facile, mais coût proportionnel au nombre de tenants. Adapté aux marchés entreprise avec des exigences de compliance strictes (santé, finance). Approche 2 — Schéma séparé par tenant : une DB partagée mais avec un schéma PostgreSQL par tenant. Bon équilibre mais complexifie les migrations. Approche 3 —

Table partagée avec colonne `tenant_id` : toutes les tables partagées, isolation via `tenant_id` et RLS. La plus efficace économiquement, adoptée par la majorité des SaaS. C'est l'approche de Social Seller.

1.4 Choisir la bonne approche

Pour un SaaS early-stage comme Social Seller, l'approche table partagée avec `tenant_id` est clairement la meilleure : coût minimal, complexité opérationnelle faible, et PostgreSQL avec RLS offre une isolation robuste. Si un jour un client entreprise exige une isolation totale pour des raisons de compliance, la migration vers une base dédiée reste possible — mais ce n'est pas le problème à résoudre au démarrage.

Isolation des données avec RLS

2.1 La colonne tenant_id

La règle d'or du multi-tenant avec table partagée : TOUTES les tables métier doivent avoir une colonne tenant_id. Pas d'exception. C'est la colonne qui détermine à quel tenant appartient chaque ligne. La colonne doit être NOT NULL, avoir une contrainte FOREIGN KEY vers la table tenants, et être indexée (elle apparaît dans tous les WHERE).

```
-- Règle : toute table métier a un tenant_id
-- Vérification automatique via une fonction utilitaire

CREATE OR REPLACE FUNCTION verify_tenant_isolation()
RETURNS TABLE(table_name TEXT, has_tenant_id BOOLEAN) AS $$
BEGIN
RETURN QUERY
SELECT
t.table_name::TEXT,
EXISTS (
SELECT 1 FROM information_schema.columns c
WHERE c.table_name = t.table_name
AND c.column_name = 'tenant_id'
) AS has_tenant_id
FROM information_schema.tables t
WHERE t.table_schema = 'public'
AND t.table_type = 'BASE TABLE'
AND t.table_name NOT IN ('tenants', 'schema_migrations', 'audit_log');
END;
$$ LANGUAGE plpgsql;

-- Résultat souhaité : has_tenant_id = true pour toutes les lignes
SELECT * FROM verify_tenant_isolation();
```

2.2 Politiques RLS multi-tenant

Row Level Security avec PostgreSQL est la protection ultime contre les bugs d'isolation. Même si le code applicatif oublie de filtrer par `tenant_id`, RLS garantit que la requête ne retournera que les données du tenant connecté. C'est une défense en profondeur : l'application filtre ET la base de données filtre.

```
-- Activer RLS sur toutes les tables métier
ALTER TABLE products ENABLE ROW LEVEL SECURITY;
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
ALTER TABLE messages ENABLE ROW LEVEL SECURITY;

-- Fonction helper : retourner le tenant_id de l'utilisateur connecté
CREATE OR REPLACE FUNCTION current_tenant_id() RETURNS UUID AS $$
SELECT tenant_id FROM users WHERE id = auth.uid();
$$ LANGUAGE sql STABLE SECURITY DEFINER;

-- Politique universelle : voir seulement son tenant
CREATE POLICY tenant_isolation ON products
USING (tenant_id = current_tenant_id());

-- Politique pour le service backend (SERVICE_ROLE bypass le RLS)
-- Le backend contourne RLS – il est responsable de son propre filtrage

-- Politique asymétrique : lire tout, écrire seulement le sien
CREATE POLICY read_all ON public_products
FOR SELECT USING (true); -- Tout le monde peut lire

CREATE POLICY write_own ON public_products
FOR ALL USING (tenant_id = current_tenant_id()); -- Écrire seulement le sien
```

2.3 Tester l'isolation

Un test d'isolation cross-tenant est le test de sécurité le plus important d'un SaaS. Il vérifie qu'un tenant ne peut pas accéder aux données d'un autre, même en manipulant les paramètres de requête. Ce test doit être automatisé et s'exécuter à chaque modification du code d'accès aux données.

```
// Test d'isolation automatisé (Vitest + Supertest)
describe('Cross-tenant isolation', () => {
  it('Tenant A cannot access Tenant B orders', async () => {
    // Créer deux tenants indépendants
    const tenantA = await createTestTenant();
    const tenantB = await createTestTenant();

    // Créer une commande dans le tenant B
    const order = await createOrder(tenantB.tenantId);
```



```
// Essayer d'y accéder depuis le tenant A
const res = await request(makeApp())
  .get(`/orders/${order.id}`)
  .set(makeAuthHeader(tenantA.tenantId, tenantA.userId));

// Doit retourner 404 – ne pas révéler l'existence
expect(res.status).toBe(404);

await deleteTestTenant(tenantA);
await deleteTestTenant(tenantB);
});
});
```

Gestion des identités et permissions

3.1 Modèle RBAC

RBAC (Role-Based Access Control) est le modèle de contrôle d'accès le plus répandu. Les permissions sont associées à des rôles, et les utilisateurs sont assignés à des rôles. Dans Social Seller, les rôles sont définis par tenant : un utilisateur peut être 'merchant' (accès total) ou 'agent' (accès limité à l'inbox et aux commandes).

```
-- Rôles dans Social Seller
-- merchant : accès complet (propriétaire de la boutique)
-- agent : accès limité (gestion inbox + commandes, pas les paramètres)

CREATE TYPE user_role AS ENUM ('merchant', 'agent', 'admin');

-- Table users avec rôle par tenant
CREATE TABLE users (
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  role user_role NOT NULL DEFAULT 'agent',
  full_name TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Middleware Express : vérifier le rôle
export function requireRole(...roles: UserRole[]) {
  return (req: Request, res: Response, next: NextFunction) => {
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({ error: 'Insufficient permissions' });
    }
    next();
  };
}

// Route réservée aux merchants
router.delete('/:id', requireRole('merchant'), async (req, res) => { ... });
```

3.2 Audit trail

Un audit trail enregistre toutes les actions sensibles avec qui a fait quoi, quand, et sur quelle donnée. Indispensable pour le debugging en production, la conformité légale, et la détection d'activité suspecte. Dans Social Seller, les changements de statut de commande et les ajustements de stock sont systématiquement loggés.

```
-- Table d'audit
CREATE TABLE audit_log (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  user_id UUID REFERENCES auth.users(id),
  action TEXT NOT NULL, -- 'order.status_changed', 'stock.adjusted'
  entity_type TEXT NOT NULL, -- 'order', 'product'
  entity_id UUID NOT NULL,
  metadata JSONB NOT NULL DEFAULT '{}',
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Exemple d'entrée d'audit
INSERT INTO audit_log (tenant_id, action, entity_type, entity_id, metadata)
VALUES (
  'tenant-abc',
  'order.status_changed',
  'order',
  'order-123',
  '{"from": "confirmed", "to": "delivered", "agent_id": "user-456"}'
);
```

■ À retenir

- ✓ Multi-tenant = plusieurs clients, une infrastructure — isolation logicielle via `tenant_id`
- ✓ Table partagée + `tenant_id` + RLS = approche optimale pour un SaaS early-stage
- ✓ Toute table métier DOIT avoir un `tenant_id` NOT NULL — sans exception
- ✓ RLS est la dernière ligne de défense — même si le code applicatif a un bug, les données restent isolées
- ✓ Les tests cross-tenant sont les tests de sécurité les plus importants du projet
- ✓ RBAC simple (merchant / agent) suffit au démarrage — complexifier selon les besoins réels

—■ Exercices

Exercice 1 — Vérification d'isolation

Auditer un schéma de base de données multi-tenant.

- Lister toutes les tables et vérifier la présence de `tenant_id`
- Vérifier que RLS est activé sur chaque table
- Écrire une politique RLS générique basée sur `current_tenant_id()`
- Tester avec deux tenants : créer des données dans l'un, vérifier l'invisibilité depuis l'autre

Exercice 2 — Système de permissions

Implémenter un RBAC à deux niveaux dans une API Express.

- Définir les rôles : `admin`, `manager`, `viewer`
- Middleware `requireRole()` qui vérifie `req.user.role`
- Appliquer les bonnes permissions sur chaque route
- Test automatisé : vérifier qu'un `viewer` ne peut pas accéder aux routes `manager`

Exercice 3 — Audit trail automatique

Créer un système d'audit automatique via un trigger PostgreSQL.

- Table `audit_log` avec les colonnes nécessaires
- Trigger AFTER INSERT/UPDATE/DELETE sur la table `orders`
- Le trigger enregistre l'ancienne et la nouvelle valeur
- Vérifier les logs après une série d'opérations

Bibliographie & Ressources

Lectures

- microsoft.com/en-us/azure/architecture/guide/multitenant/overview — guide multi-tenant Azure
- stripe.com/atlas/guides/saas-pricing — stratégies de pricing SaaS
- highscalability.com — études de cas d'architectures SaaS à grande échelle

Outils

- posthog.com — analytics product pour SaaS
- stripe.com — facturation et abonnements
- supabase.com/docs/guides/auth/row-level-security — RLS guide complet